

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 1

Student Name: K.Rama

Roll No.: A24126552087

Section: CSM-B

Date: 16-09-2026

1. TITLE

Design of a Pure Semantic HTML Static Webpage without CSS or JavaScript

2. AIM

To design a fully static webpage using only semantic HTML elements, native browser-rendered form controls, and a table-based structural layout, without using any external CSS stylesheet or JavaScript.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the role of semantic tags such as header, nav, section, main, article, aside, and footer in organizing a document.
- To build a two-column page layout using an HTML table in the absence of CSS.
- To use native HTML form controls (text, email, select, textarea, submit, reset) without any scripting.
- To observe how browsers render unstyled markup by default.
- To appreciate the accessibility and performance benefits of relying on browser defaults.

4. THEORY

A pure HTML document relies entirely on the browser's built-in user-agent stylesheet and native rendering rules, since no external CSS or JavaScript is loaded. Semantic elements such as <header>, <nav>, <section>, <main>, <article>, <aside>, and <footer> do not add any visual styling by themselves, but they communicate the structural role of each block to browsers, screen readers, and search engine crawlers. In the absence of CSS-based layout tools such as Flexbox or Grid, a common way to arrange content into columns is the HTML <table> element, using width and valign attributes on <td> cells to approximate a layout. Native form controls — <input>, <select>, and <textarea> — are rendered by the browser with default borders, fonts, and spacing, and attributes like required and placeholder provide basic validation and guidance without any JavaScript. Because there is no scripting, the submit and reset buttons rely entirely on the browser's built-in form-handling behaviour.

5. ALGORITHM / PROCEDURE

1. Create a single index.html file and declare the DOCTYPE, html, head, and body elements.
2. Inside <header>, use a two-cell HTML table to place the site heading on the left and a horizontal <nav> menu of anchor links on the right.
3. Add a <hr> separator, then a <section id="home"> containing a centred heading, an introductory paragraph, and a call-to-action link, wrapped in <center> tags.
4. Add another <hr>, then build the main two-column body using a structural <table> with one row and two <td> cells (70% and 30% width).

5. In the left <td>, add a <main id="articles"> containing two <article> blocks, each with a heading, a publish date paragraph, and descriptive paragraphs using inline <code> tags for keywords.
6. Inside the second article, add a <form id="contact"> wrapped in a <fieldset> with a <legend>, containing labelled text, email, select, and textarea controls plus submit and reset buttons.
7. In the right <td> (shaded with bgcolor), add an <aside id="about"> with a heading, an explanatory paragraph, an unordered list of benefits, and an ordered list of required components.
8. Add a final <hr> and a <footer> containing a centred copyright paragraph.
9. Open index.html directly in a browser and verify that the header, navigation, hero section, two-column layout, form, and footer all render correctly using only browser defaults.

6. CODE EXPLANATION

The entire page lives in a single index.html file with no external stylesheet or script. The <header> uses a borderless HTML table purely for layout, placing the "StaticWeb" heading and a pipe-separated <nav> menu side by side. The <section id="home"> acts as a hero area, centred using the legacy <center> tag since no CSS is available. The page body is split into two columns with a second structural table: the left 70%-wide cell holds <main id="articles">, which contains two <article> elements — one describing semantic tags and one containing a fully native HTML <form> with text, email, dropdown, and textarea inputs plus submit/reset buttons, all grouped inside a <fieldset> with a <legend>. The right 30%-wide cell holds <aside id="about"> with a shaded background (via the bgcolor attribute), listing benefits in a and required components in an . A <footer> with a centred copyright notice closes the page. Because there is no CSS, every visual detail — spacing, borders on the fieldset, list bullets and numbers — comes directly from the browser's default rendering of these semantic and form elements.

7. PROGRAM

index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>Pure Semantic HTML Static Webpage</title>
```

```
</head>
```

```
<body>
```

```
<!-- Header Section -->
```

```
<header>
```

```
<table width="100%" border="0" cellspacing="0" cellpadding="10">
```

```
<tr>
```

```
<td>
```

```
<h1>StaticWeb</h1>
```

```
</td>
```

```
<td align="right">
```

```
<nav>
```

```

<a href="#home">Home</a> &nbsp;&nbsp;&nbsp;|&nbsp;&nbsp;&nbsp;
<a href="#articles">Articles</a> &nbsp;&nbsp;&nbsp;|&nbsp;&nbsp;&nbsp;
<a href="#about">About</a> &nbsp;&nbsp;&nbsp;|&nbsp;&nbsp;&nbsp;
<a href="#contact">Contact Us</a>
</nav>
</td>
</tr>
</table>
</header>

<hr>

<!-- Hero / Showcase Section -->
<section id="home">
<center>
<h2>Semantic Structures Without CSS</h2>
<p>This layout uses raw HTML components to build an organized document hierarchy
that browsers can cleanly parse out-of-the-box.</p>
<p><a href="#articles"><b>Read Our Articles Below &darr;</b></a></p>
</center>
</section>

<hr>

<!-- Main Content Layout using a Structural Table -->
<table width="100%" border="0" cellspacing="0" cellpadding="20">
<tr valign="top">

<!-- Left Side: Main Content Column -->
<td width="70%">
<main id="articles">

<article>
<h2>1. Core Structural Tags</h2>
<p>Published: July 7, 2026</p>
<p>When you omit cascading style sheets entirely, the semantic markup handles all of
the structural heavy lifting. Elements like <code>&lt;main&gt;</code>,
<code>&lt;article&gt;</code>, and <code>&lt;section&gt;</code> tell screen readers
and web scrapers exactly how information flows.</p>
<p>Using raw browser defaults ensures excellent loading performance, perfect
accessibility compliance, and absolute structural clarity.</p>
</article>

<br><hr><br>

<article>
<h2>2. Native Interaction Elements</h2>
<p>Published: July 5, 2026</p>

```

<p>Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.</p>

<h3>Interactive Demonstration Forms</h3>

```
<form id="contact" action="#" method="post">
<fieldset>
<legend><b>User Inquiry Form</b></legend>
<p>
<label for="name">Full Name: </label>
<input type="text" id="name" name="username" required placeholder="John Doe">
</p>
<p>
<label for="email">Email Address: </label>
<input type="email" id="email" name="useremail" required>
</p>
<p>
<label for="topic">Inquiry Topic: </label>
<select id="topic" name="usertopic">
<option value="general">General Feedback</option>
<option value="technical">Technical Architecture</option>
<option value="academics">Academic Curriculum</option>
</select>
</p>
<p>
<label for="message">Message Details:</label><br>
<textarea id="message" name="usermessage" rows="5" cols="40" placeholder="Type
your notes here..."></textarea>
</p>
<p>
<input type="submit" value="Submit Form Data">
<input type="reset" value="Clear Entries">
</p>
</fieldset>
</form>
</article>

</main>
</td>
```

<!-- Right Side: Sidebar Column -->

<td width="30%" bgcolor="#f4f4f4">

<aside id="about">

<h3>Document References</h3>

<p>Static pages map data out directly without real-time database queries or processor overhead.</p>

<h4>Core Benefits:</h4>

```

</ul>
<li>Instant browser painting</li>
<li>Low memory footprints</li>
<li>High data compatibility</li>
</ul>

<h4>Required Components:</h4>
<ol>
<li>Document Type Definition</li>
<li>Semantic Wrapper Blocks</li>
<li>Data Input Control Interfaces</li>
</ol>
</aside>
</td>

```

```

</tr>
</table>

```

```

<hr>

```

```

<!-- Footer Section -->
<footer>
<center>
<p>&copy; 2026 Pure HTML Architecture. Assembled completely without style
configurations.</p>
</center>
</footer>

```

```

</body>
</html>

```

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Open index.html in a browser	Header, nav links, hero section, two-column layout, and footer all display using browser defaults	Page rendered correctly with all sections visible	Pass
TC-02	Click a navigation link (e.g. "Articles")	Browser jumps to the section with the matching id	Page scrolled to the correct anchor	Pass
TC-03	Submit the form with the "Full Name" field left empty	Browser's built-in required validation blocks submission and	Validation message shown, form not submitted	Pass

		prompts the user		
TC-04	Fill in Name, Email, Topic, and Message, then click "Submit Form Data"	Browser attempts to submit the form data via POST to the action URL	Form data collected and submission attempted	Pass
TC-05	Fill in form fields and click "Clear Entries"	All input fields are reset to their default empty state	Form fields cleared successfully	Pass

9. OUTPUT

Rendered page (header, hero section, articles, and sidebar):

Semantic Structures Without CSS

This layout uses raw HTML components to build an organized document hierarchy that browsers can cleanly parse out-of-the-box.

[Read Our Articles Below ↓](#)

1. Core Structural Tags

Published: July 7, 2026

When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like `<main>`, `<article>`, and `<section>` tell screen readers and web scrapers exactly how information flows.

Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.

2. Native Interaction Elements

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic:

Message Details:

Type your notes here...

Document References

Static pages map data out directly without real-time database queries or processor overhead.

Core Benefits:

- Instant browser painting
- Low memory footprints
- High data compatibility

Required Components:

1. Document Type Definition
2. Semantic Wrapper Blocks
3. Data Input Control Interfaces

10. RESULT

Thus, a fully static webpage was successfully designed using only pure semantic HTML elements, a table-based structural layout, and native browser form controls, without using any external CSS or JavaScript. The page rendered correctly with a working header, navigation, two-column content layout, an interactive form, and a footer, all styled entirely by browser defaults.